p0750r1
Consume

Published Proposal, 11 February 2018

This version:
http://wg21.link/P0750

Authors:
JF Bastien (Apple)
Paul E. McKenney (IBM)

Audience:
SG1

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
github.com/jfbastien/papers/blob/master/source/P0750r1.bs

Implementation:
github.com/jfbastien/stdconsume

Abstract

Fixing memory order consume.

Table of Contents

§ 1. Edit History

§ 1.1. r0 → r1

Minor editorial fixes.

[p0750r0] was discussed by SG1 in Albuquerque, there's optimism on finding a solution along the lines presented in the paper, but more work needs to be put in the prevent value speculation by the compiler.

§ 2. Background

Memory order consume as specified in C++11 hasn't been implemented by any compiler. The main reason for this shortcoming is that the specification creates a fundamental mismatch between the model of "dependency" at the C++ source code level and "dependency" at the ISA-specific assembly level.

Programmers targeting large systems have been using "consume" under different names since at least the mid-90's. As hardware evolved, what used to be an esoteric memory detail of large systems has become commonplace. For example, Linux gained support for Read Copy Update [RCU] in October 2002 and usage has steadily increased since then [RCUUsage]. Memory order consume is the key to RCU's scalability.

This proposal continues work from:

1. Towards Implementation and Use of `memory_order_consume` [p0098R1] lists a plethora of manners in which C++ could expose consume. In particular, section 7.11 is the root of the current proposal.

2. Marking `memory_order_consume` Dependency Chains [p0462r1] expands on the previous paper. Of particular interest is section 3.2 which proposes `depending_ptr`, which the current proposal's API derives from.
3. Proposal for New `memory_order_consume` Definition [p0190r3] has comprehensive details on each operation which RCU supports. That list was critical in crafting the precise semantics which the current proposal has.

The current proposal is different from prior proposals in that it contains a single proposed API, which is known to work for at least one code base. Indeed, it is based on work in WebKit [Atomics] which has been successfully using consume memory ordering in C++ on ARM platforms for over a year.

The WebKit API was written by low-level compiler experts for their own use, with limited applicability compared to RCU, and isn't necessarily user-friendly. The current proposal therefore differs as follows:

- Fewer sharp edges than the WebKit one; and
- Attempt to support more usecases as documented by [p0462r1]

§ 3. What is `consume`?

Consume exposes low-level architecture dependencies as available in weak memory multicore systems. For example, it maps to what ARM calls the "address dependency rule" as explained in section 6.3 of the ARM Barrier Litmus Tests and Cookbook [Litmus]. POWER has a similar ISA feature. On weak memory ISAs, consume allows writing release / acquire style code, where the reader side doesn't need extra fences. Ordering is guaranteed by creating dependencies on the reader side between the released location and the subsequent reads that should be observed to happen after that location's value was written by the writer. A "dependency" at the ISA level means that computation of the value of the address of subsequent loads depend on the value loaded from the release location.

It is always valid for an implementation to "promote" consume to acquire, but in most cases this adds an extra fence.

Despite being used extensively in multiple systems, crafting a consume API which is easy to use correctly remains a difficult task because it requires very precise assembly generation. To maintain correctness, current uses rely on:

- A gentleperson's agreement with compiler authors to avoid breaking code;
- Human inspection of compiled code disassembly;
- Extensive testing;

- Judicious application of inline assembly; and
- Luck 🍀.

This proposal is no different, except that it offloads this burden on the compiler and standard library authors. These people are in a much better position to ensure correctness: they rely on more engineering than luck to maintain the language's correctness because it's literally the main purpose of their software.

§ 4. Wording as of C++17

> ¶7 An evaluation A carries a dependency to an evaluation B if
>
> 1. the value of A is used as an operand of B, unless:
>   1. B is an invocation of any specialization of `std::kill_dependency`, or
>   2. A is the left operand of a built-in logical AND `&&` or logical OR `||` operator, or
>   3. A is the left operand of a conditional `?:` operator, or
>   4. A is the left operand of the built-in comma `,` operator; or
> 2. A writes a scalar object or bit-field M, B reads the value written by A from M, and A is sequenced before B, or
> 3. for some evaluation X, A carries a dependency to X, and X carries a dependency to B.
>
> [Note: "Carries a dependency to" is a subset of "is sequenced before", and is similarly strictly intra-thread. — end note]
>
> ¶8 An evaluation A is dependency-ordered before an evaluation B if
>
> 1. A performs a release operation on an atomic object M, and, in another thread, B performs a consume operation on M and reads a value written by any side effect in the release sequence headed by A, or
> 2. for some evaluation X, A is dependency-ordered before X and X carries a dependency to B.
>
> [Note: The relation "is dependency-ordered before" is analogous to "synchronizes with", but uses release/consume in place of release/acquire. — end note]

The above wording and subsequent uses of it terms need to be updated.

§ 5. Other considerations

The following can also be considered as part of this proposal:

1. Deprecate passing `memory_order_consume` to the existing variable functions.
2. Make atomic ordering a template parameter tag instead of a variable. This would be a huge change, maybe we should do it separately, but it would simplify this proposal if we had it.
3. Deprecate attribute `[[carries_dependency]]`.
4. C compatibility.

The author is happy to add these to the current proposal if SG1 thinks it would be useful to explore.

§ 6. Future work

The current proposal has a rough API, very rough wording, and a small number of tests. More work needs to go towards these items. Furthermore, the following operations need to be added:

- xchg
- cmpxchg
- read-modify-write
- dependent member pointers, returning `&(ptr1->*ptr2)`

It seems like the easiest way to go about is to try out the API in a from-scratch C++ codebase. An obvious candidate would be a user-space RCU implementation in C++ using this API.

It is also worth considering whether consumable types should be restricted somehow. `atomic` requires trivially-copyable types, but doesn't restrict the size of `T`. Should consume require the same restriction? Is it useful to consume very large values? In the authors' experience it isn't, and the Linux kernel consumes only pointers, but it could be useful in other fields, and at a minimum what qualifies as "large" is ISA-dependent.

Generated assembly needs to be inspected on multiple implementations to make sure various compilers and ISAs can support the proposed design. Already the sample implementation has been observed spilling dependencies to the stack on ARM64 using GCC, which happens to be valid but might be brittle.

§ 7. Proposed API

The following is a proposed API, a full implementation and tests are available on GitHub under [StdConsume].

```
class dependency;

// A value and its dependency.
// FIXME: should this be opaque, to allow T to carry the dependency implici
template<typename T, typename Dependency = dependency>
struct dependent {
    T value;
    Dependency dependency;

    dependent() = delete;
    dependent(T value, Dependency dependency) : value(value), dependency(de
    dependent(T value) : value(value), dependency(value) {}
};

// A dependent_ptr contains a pointer which was obtained through a consume
// operation. It supports a restricted set of operations compared to regula
// pointers, which allows it to continue carrying its dependency.
//
// dependent_ptr differs from dependent<T*> by having a single data member,
// by acting similarly to how regular pointers act. A dependent_ptr is a us
// abstraction because it closely matches the low-level details of modern
// ISA-specific dependencies.
template<typename T>
class dependent_ptr {
public:
    // Constructors

    // No dependency yet.
    dependent_ptr() = default;
    dependent_ptr(T*); // FIXME: should this automatically create a depende
    dependent_ptr(std::nullptr_t);

    // With dependency.
    dependent_ptr(T*, dependency);
    dependent_ptr(std::nullptr_t, dependency);
    dependent_ptr(dependent<uintptr_t>);
    dependent_ptr(dependent<intptr_t>);

    // Copy construction extends the right-hand side chain to cover both
    // dependent pointers. The left-hand side chain is broken.
    dependent_ptr(const dependent_ptr&);

    // Moving, Copying, and Casting
```

```
    // Assigning a non-dependent right-hand side breaks the left-hand side'
    // chain.
    dependent_ptr& operator=(T*);
    dependent_ptr& operator=(std::nullptr_t);

    // Using a dependent pointer as the right-hand side of an assignment
    // expression extends the chain to cover both the assignment and the va
    // returned by that assignment statement.
    dependent_ptr& operator=(const dependent_ptr&);

    // If a pointer value is part of a dependency chain, then converting it
    // intptr_t or uintptr_t extends the chain to the result's dependency.
    // can be used to perform pointer tagging (with usual C++ caveats on po
    // tagging) while retaining dependencies.
    dependent<uintptr_t> to_uintptr_t() const;
    dependent<intptr_t> to_intptr_t() const;

    // Pointer Offsets

    // Indexing though a dependent pointer extends the chain to the resulti
    // value.
    dependent<T> operator[](size_t offset) const;

    // Class-member access operators can be thought of as computing an offs
    // access itself is in the dependency chain, but such access does not e
    // the chain to cover the result.
    T* operator->() const;

    // Dereferencing and Address-Of

    // Dereferencing a dependent pointer extends the chain to the resulting
    dependent<T> operator*() const;

    // If a pointer is part of a dependency chain, then applying the unary
    // address-of operator extends the chain to the result.
    dependent_ptr<T*> operator&() const;

    // In some circumstances, such as for function pointers, the raw pointe
    // value is required. The chain extend to that value.
    T* value() const;

    // A pure dependency from the dependent_ptr.
```

```
    dependency dependency() const;

    // Comparisons aren't needed because the T* themselves can be compared
    // without breaking the dependency chain of the dependent_ptr. This is
    // important because compilers could optimize certain accesses based or
    // result of comparisons, breaking explicitly constructed chains in the
    // process.

private:
    // Exposition only:
    T* ptr { nullptr };
};

// A dependency is an opaque value which can be chained through consume
// operations. Chaining dependencies ensures that load operations carry
// dependencies between each other. Dependencies can also be combined to cr
// a new dependency which implies a dependency on the combined inputs.
//
// [ Note: Dependencies create false dependencies as defined by existing IS
//   — end note ]
class dependency {
public:
    template<typename T> dependency(T);
    dependency() = delete;

    // Dependency combination.
    dependency operator|(dependency d);
    template<typename T> friend dependency operator|(dependency, dependent_
    template<typename T> friend dependency operator|(dependent_ptr<T>, depe

    // Pointer tagging.
    friend uintptr_t operator|(dependency, uintptr_t);
    friend uintptr_t operator|(uintptr_t, dependency);
    friend intptr_t operator|(dependency, intptr_t);
    friend intptr_t operator|(intptr_t, dependency);

    // Exposition only:
    typedef unsigned dependency_type;

private:
    // Exposition only:
    dependency_type dep;
};
```

```
// Free-function dependency combination.
template<typename T> dependency operator|(dependency, dependent_ptr<T>);
template<typename T> dependency operator|(dependent_ptr<T>, dependency);

// Free-function pointer tagging with dependency.
uintptr_t operator|(dependency, uintptr_t);
uintptr_t operator|(uintptr_t, dependency);
intptr_t operator|(dependency, intptr_t);
intptr_t operator|(intptr_t, dependency);

// Beginning of dependency chain.
template<typename T> dependent_ptr<T> consume_load(const std::atomic<T*>&);
template<typename T> dependent<T> consume_load(const std::atomic<T>&);

// Subsequent dependent operations.
template<typename T> dependent_ptr<T> consume_load(dependent_ptr<T*>);
template<typename T> dependent<T> consume_load(dependent_ptr<T>);
template<typename T> dependent_ptr<T> consume_load(T**, dependency);
template<typename T> dependent<T> consume_load(T*, dependency);
```

§ 8. Acknowledgements

The Committee's SG1 sub-group has had many lengthy discussions about memory order consume, and how to fix it. It took many failed attempts to come to this solution. In particular, the following people were tremendously helpful in reviewing this paper: Hans Boehm, Olivier Giroux, Paul McKenney, Robin Morisset, Tim Northover, Will Deacon. Filip Pizło contributed substantially to the WebKit implementation and usage of consume, including the first valuable usecase which I saw as a perfect excuse to try out a consume solution that I had in mind.

§ References

§ Informative References

[Atomics]

JF Bastien; Filip Jerzy Pizło. WebKit source: WTF Atomics.h. June 2, 2017. URL: https://trac.webkit.org/browser/webkit/trunk/Source/WTF/wtf/Atomics.h?rev=+217722#L342

[Litmus]

ARM Limited. ARM Barrier Litmus Tests and Cookbook. November 26, 2009. URL: http://infocenter.arm.com/help/topic/com.arm.doc.genc007826/Barrier_Litmus_Tests_and_Cookbook_A08.pdf

[p0098R1]

Paul E. McKenney, Torvald Riegel, Jeff Preshing, Hans Boehm, Clark Nelson, Olivier Giroux, Lawrence Crowl. Towards Implementation and Use of memory order consume. 4 January 2016. URL: https://wg21.link/p0098r1

[P0190R3]

Paul E. McKenney, Michael Wong, Hans Boehm, Jens Maurer, Jeffrey Yasskin, JF Bastien. Proposal for New memory order consume Definition. 5 February 2017. URL: https://wg21.link/p0190r3

[P0462R1]

Paul E. McKenney, Torvald Riegel, Jeff Preshing, Hans Boehm, Clark Nelson, Olivier Giroux, Lawrence Crowl, JF Bastien, Micheal Wong. Marking memory order consume Dependency Chains. 5 February 2017. URL: https://wg21.link/p0462r1

[P0750R0]

JF Bastien. Consume. URL: https://wg21.link/p0750r0

[RCU]

Paul McKenney. What is RCU?. August 17, 2017. URL: https://www.kernel.org/doc/Documentation/RCU/whatisRCU.txt

[RCUUsage]

Paul McKenney. RCU Linux Usage. July 2, 2017. URL: http://www.rdrop.com/users/paulmck/RCU/linuxusage.html

[StdConsume]

JF Bastien. GitHub: jfbastien / stdconsume. October 9, 2017. URL: https://github.com/jfbastien/stdconsume